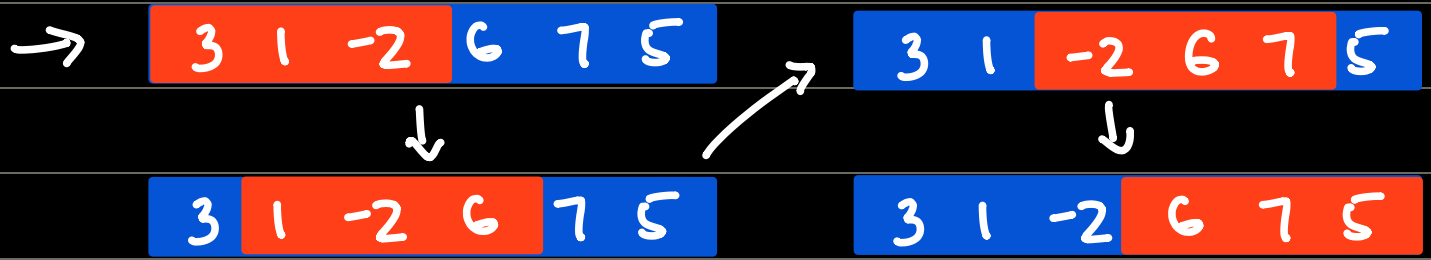


Sliding Window

→ A way of efficiently processing a subset of data, by focusing only on the most relevant portion.



Types

Fixed → Maintains a constant length as it slides across the ds. Used when the window size is known in advance, when we have been asked to find a subarray/string of a given length.

Variable → Window size is not fixed, it changes based on a condition.

Fixed Template

`left = 0`

`window_size = initialize`

`for right in range(len(array)):`

`add array[right] to window_stack`

`if (right - left + 1) == k:`

`record/update answer using window_stack`

`remove array[left] from window_stack`

left += 1

[2, 1, 5, 1, 3, 2] k=3
↑

right sum=2, window size=1, not yet k

[2, 1, 5, 1, 3, 2] k=3
↑

right sum=3, window size=2, not yet k

[2, 1, 5, 1, 3, 2] k=3
↑

right sum=8, window size=3, equals k. record sum. remove array[0] (2), sum now becomes 6, left becomes 1.

[2, 1, 5, 1, 3, 2] k=3
↑ ↑

left right sum=7, window size=3, equals k. record sum. remove array[1] (1), sum now becomes 6, left becomes 2.

[2, 1, 5, 1, 3, 2] k=3
↑ ↑

left right sum=9, window size=3, equals k. record sum. remove array[2] (5), sum now becomes 4, left becomes 3.

$[2, 1, 5, 1, 3, 2]$ $k=3$
 $\uparrow$ $\uparrow$
left right

sum = 6, window size = 3, equals k. record sum.

That's it. The max recorded sum was 9. Each element is added exactly once & removed exactly once.
→ $O(n)$ instead of $O(n \text{ times } k)$.

Variable template

left = 0

window_stack = initialize

for right in range(len(array)):

 add array[right] to window_stack

 while window_stack violates the condition:

 remove array[left] from window_stack

 left += 1

 update answer using (right - left + 1)

→ Right & left only ever moves forward.

→ Even though this has nested loops, the complexity is $O(n)$, because the inner while loop across the entire run of the algorithm can only execute n times total across all iterations.

- right moves exactly once per iteration of the for loop. the for loop runs n times. So right moves forward n times.
- left moves forward inside the while loop. Starts at 0 and can only ever go up to $n-1$. Never resets. No matter how many times the while loop runs, left cannot move forward more than n times total.
- How many times does the while loop run across ALL iterations of the for loop combined
- right $\rightarrow n$, left $\rightarrow n$, total $\rightarrow 2n$
constant dropped $\rightarrow$ becomes $O(n)$

Variable Size window

- Looking for longest window satisfying a condition.
 - ↳ Expand right always. Left is shrinked only while the window is invalid. After shrinking, the best answer is updated. Right now you have a valid window & you want to know if it's the biggest.
- Looking for shortest window satisfying a condition.
 - ↳ Expand right until the window becomes valid. Then, while it's valid, we shrink from the left to see how small you can make it, till it breaks, updating the best answer at each successful shrink.

window-size depends on what the problem is.
Could be a running sum or a frequency map
or a count of distinct elements.

Problem to solve

Array = [2, 3, 1, 2, 4, 3] Target = 7

→ Expand right until the condition becomes valid.

$2 < 7 \times$ $2, 3 < 7 \times$ $2, 3, 1 < 7 \times$
 $2, 3, 1, 2 > 7 \checkmark$

→ while it's valid, we shrink from the left till it breaks.

$2, 3, 1, 2 > 7 \rightarrow 3, 1, 2 < 7 \times$

→ Therefore, **2, 3, 1, 2** is our best answer now.

→ Expand right again. 2, 3, 1, 2, 4

→ Now, we shrink from left. $3, 1, 2, 4 > 7$

→ Then, we shrink from left again. $1, 2, 4 == 7$

→ **1, 2, 4** is now our best answer.

→ Expand right again. 1, 2, 4, 3

→ Now, shrink from left. $2, 4, 3 > 7$

→ Then, shrink again. $4, 3 == 7$

→ **4, 3** is our final best answer.